Viviane Flor Park
169259

Implementação de Compilador C-

São José dos Campos
2026

Viviane Flor Park
169259

Implementação de Compilador C-

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina Laboratório de Sistemas Computacionais: Compiladores.

Docente: Prof. Dr. Luiz Eduardo Galvão Martins

Universidade Federal de São Paulo

Instituto de Ciência e Tecnologia

São José dos Campos

2026

Sumário

1 INTRODUÇÃO 4

2 PROCESSADOR 5
2.1 Visão Geral 5
2.2 Unidades Funcionais 6
2.2.1 *Program Counter* 6
2.2.2 Memória de Instruções e Unidade de Decodificação 6
2.2.3 Banco de Registradores 7
2.2.4 Unidade Lógica e Aritmética 7
2.2.5 Memória Principal 7
2.2.6 *Current Program Status Register* e Unidade de Validação 7
2.2.7 Unidade de Controle 8
2.2.8 Seletor de *Clock*, Entrada e Saída 8
2.3 Conjunto de Instruções 8
2.3.1 Instruções de Tipo D 9
2.3.2 Instruções de Tipo B 10
2.3.3 Instruções de Tipo M 10
2.3.4 Instruções de Tipo A 10
2.4 Organização da Memória 10

3 COMPILADOR: FASE DE ANÁLISE 11
3.1 Modelagem 11
3.2 Análise Léxica 11
3.3 Análise Sintática 11
3.4 Análise Semântica 11

4 COMPILADOR: FASE DE SÍNTESE 12
4.1 Modelagem 12
4.2 Geração de Código Intermediário 12
4.3 Geração de Código Assembly 12
4.4 Geração de Código Executável 12
4.5 Gerenciamento de Memória 12

5 EXEMPLOS 13
5.1 Código Fonte 13
5.2 Código Intermediário 13

5.3 Código Assembly 13
5.4 Código Executável 13
5.5 13

6 CONCLUSÃO 14

REFERÊNCIAS 15

1 Introdução

2 Processador

2.1 Visão Geral

O processador utilizado para o desenvolvimento deste projeto implementa uma arquitetura ARM monociclo adaptada de 32 bits que segue os padrões da arquitetura de Harvard. Nessa implementação, a memória de programa guarda uma palavra de 32 bits por linha e o banco de registradores conta com 32 registradores de propósito geral, todos de 32 bits. Entre os registradores de propósito geral, o registrador R31 foi reservado para uso como *link register* (LR). Também foram projetados dois registradores de uso dedicado: o *program counter* (PC), de 32 bits, e o *current program status register* (CPSR), de 4 bits. O processador também conta com um sistema de interface com um *field programmable gate array* (FPGA).

O diagrama de blocos do processador, observado na Figura 2, mostra as diferentes unidades funcionais, como PC, memória de instruções, unidade de decodificação, banco de registradores, unidade lógica e aritmética (ULA), CPSR, memória de dados, unidade de validação, unidade de controle, multiplexadores e *adder*, e como elas interagem entre si. Estão presentes no diagrama também os módulos do sistema de interface com FPGA, como entrada, saída, seletor de *clock* e unidade de *debouncing*. Detalhes a respeito dos propósitos, implementação, funcionamento e interligação de cada uma dessas unidades podem ser obtidos na seção 2.2.

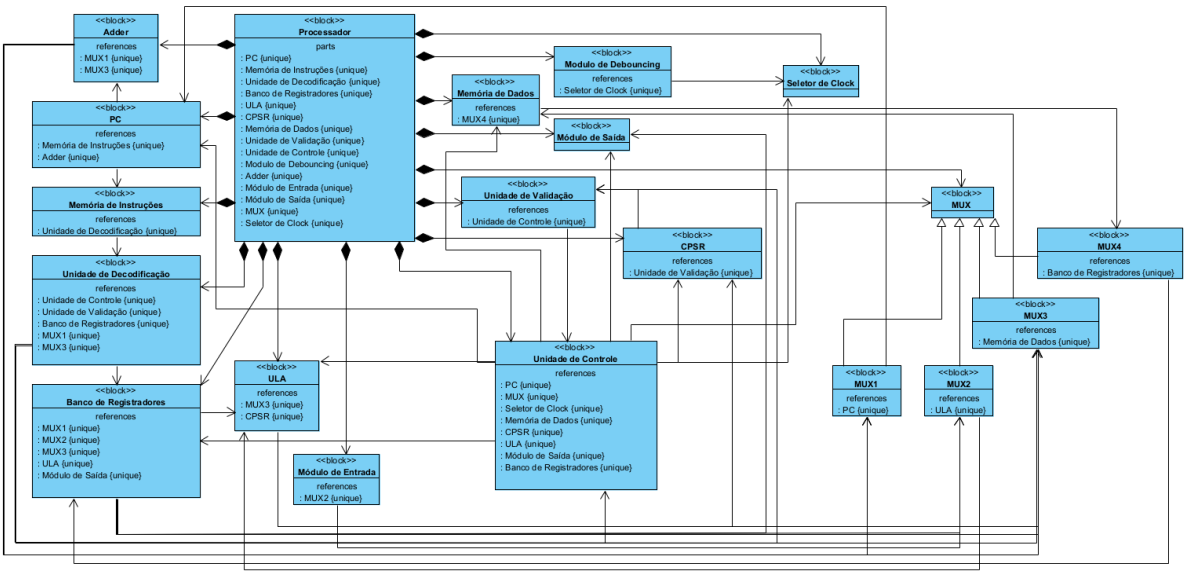


Figura 1 – Diagrama de blocos do processador. Fonte: A autora (2026).

2.2 Unidades Funcionais

Para uma visualização mais clara da maneira como as diferentes unidades funcionais do processador se interligam e como ocorre o fluxo de dados, é apresentado na Figura 2 uma visão geral da estrutura do processador.

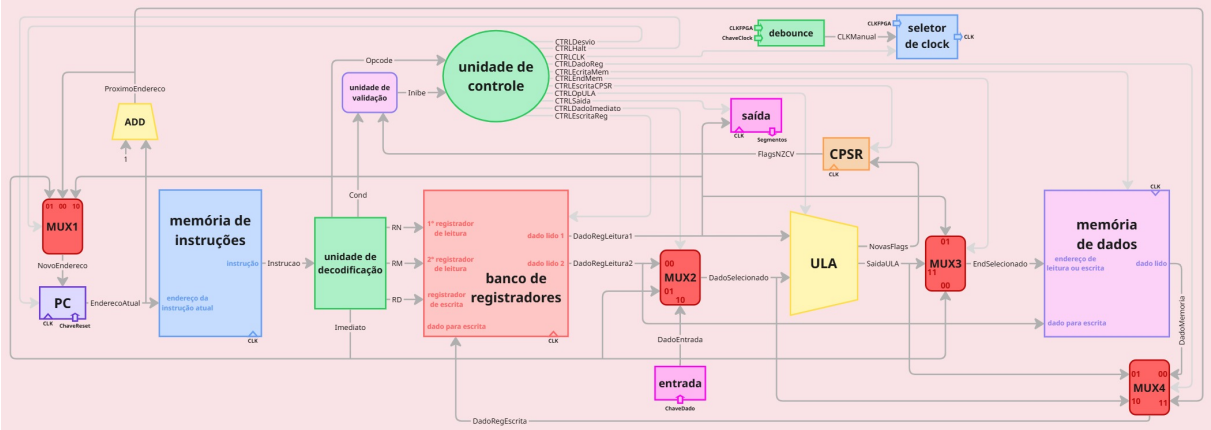


Figura 2 – Visão geral da estrutura processador. Fonte: A autora (2025).

2.2.1 *Program Counter*

O PC é um registrador especial que guarda o endereço da instrução executada. A cada borda de subida do *clock*, essa informação é atualizada seja pelo incremento unitário do *adder*, por um endereço contido em um registrador qualquer do banco de registradores ou por um endereço de desvio contido em uma instrução. Essa decisão é feita por um multiplexador (nesse caso, o MUX1), conforme visto na Figura 2, com o auxílio do sinal CTRLDesvio advindo da unidade de controle.

A atualização desse registrador pode ser inibida pelo sinal de controle CTRLHalt, emitido pela unidade de controle diante de uma instrução HLT, conforme explicitado na seção 2.3. Além disso, seu conteúdo pode ser resetado para zero por meio da entrada ChaveReset, que foi implementada como uma chave de uma placa FPGA.

2.2.2 Memória de Instruções e Unidade de Decodificação

O conteúdo do PC é alimentado à memória de instruções, que lê a instrução contida naquele endereço e a disponibiliza em sua saída. Esse processo é realizada de forma síncrona, com a borda de descida do *clock*.

A instrução, então, é fornecida à unidade de decodificação, que identifica o seu tipo, conforme descrito na seção 2.3, e a particiona nos campos correspondentes.

2.2.3 Banco de Registradores

O banco de registradores recebe dois endereços de leitura e um de escrita, além do dado para a escrita. A leitura dos registradores é feita de forma assíncrona e disponibilizada prontamente nas saídas do banco, conforme visto na Figura 2. A escrita em registradores, por outro lado, é feita de forma síncrona com a borda de subida do *clock* de acordo com o sinal de controle CTRLEscritaReg.

Os dados lidos do banco de registradores são disponibilizados para diversas outras unidades funcionais do processador, como para o próprio PC, conforme já descrito anteriormente, para a ULA, memória de dados ou módulo de saída. Esses dados podem até ser realimentados para o próprio banco.

2.2.4 Unidade Lógica e Aritmética

A ULA tem duas funções principais: realizar operações lógicas e aritméticas e, com base nestas, fazer o cálculo das novas *flags* NZCV do CPSR, conforme descrito na seção 2.2.6. Ela recebe dois operandos, que podem vir do banco de registradores ou da instrução, além do sinal CTRLOpULA, vindo da unidade de controle, que decide qual operação será realizada sobre os dados. A ULA pode realizar 9 operações distintas: soma, subtração (de duas maneiras diferentes, invertendo a ordem dos operandos), multiplicação, divisão, *not*, *and*, *or* e *xor*.

Os resultados da operações realizadas por ela podem ser alimentados à memória principal ou ao banco de registradores. As *flags* calculadas são sempre fornecidas ao CPSR.

2.2.5 Memória Principal

A memória de dados recebe um endereço que pode ser tanto de leitura, quanto de escrita. O conteúdo contido nesse endereço sempre é disponibilizado em sua saída de maneira assíncrona. Já a escrita de um dado neste endereço é feita de forma síncrona com a borda de subida do *clock*, de acordo com o sinal de controle CTRLEscritaMem.

O endereço fornecido à memória pode ter origem na instrução, no caso de endereçamento direto, no banco de registradores, no caso de endereçamento indireto por registrador, ou no resultado de uma operação da ULA, para o endereçamento por deslocamento. A seção 2.3.3 discute mais sobre as diferentes formas de endereçamento à memória às quais o processador oferece suporte.

2.2.6 *Current Program Status Register* e Unidade de Validação

O CPSR e a unidade de validação são, juntas, o coração da execução condicional de instruções, que é o diferencial da arquitetura ARM.

O CPSR é um registrador especial que guarda as *flags* NZCV, quatro bits de informação que dizem respeito ao valor do resultado da ULA. Essa informação consiste nas *flags* N (*negative*), que indica se o resultado é negativo, Z (*zero*), que indica se o resultado é zero, C (*carry*), que indica se o resultado gerou um bit de *carry*, e V (*overflow*), que indica se o resultado estourou seu tamanho máximo.

Conforme visto na Figura 2, as informações do CPSR são alimentadas à unidade de validação junto a um campo especial da instrução, chamado de *cond*. Essas informações são comparadas e, então, é gerado um sinal que decide se a execução da instrução deve ou não ser inibida. A correspondência entre essas informações é discutida com mais detalhes na seção 2.3.

2.2.7 Unidade de Controle

A unidade de controle recebe o *opcode* da instrução e decide, então, a configuração dos sinais de controle. Além disso, caso a unidade de validação tenha gerado um sinal positivo de inibição, ela trata a instrução atual como uma instrução NOP.

2.2.8 Seletor de *Clock*, Entrada e Saída

O seletor de *clock* e os módulos de entrada e saída fazem parte do sistema de interface com FPGA do processador. O primeiro realiza a divisão de frequência do *clock* interno de 50MHz do FPGA, tornando a frequência do *clock* padrão que rege o processamento 1MHz, além de ter o poder de alterar esse *clock* para uma chave da placa, de forma que funcione de maneira manual. Essa segunda funcionalidade do seletor de *clock* é utilizada em instruções IN e OUT.

O módulo de entrada lê um dado codificado em binário nas chaves do FPGA e o disponibiliza para processamento posterior. Ele é utilizado em instruções IN, que guardam essa informação em um registrador. Já o módulo de saída recebe a informação em binário de um registrador e faz sua conversão de modo que possa ser visualizado nos *displays* de 7 segmentos do FPGA.

2.3 Conjunto de Instruções

O conjunto de instruções ao qual o processador oferece suporte pode ser subdividido em quatro categorias: instruções de processamento de dados ou tipo D, de desvio ou tipo B, de acesso à memória ou tipo M e alternativas ou tipo A.

Todas as instruções têm os 6 bits mais significativos em comum. Os quatro primeiros compõem o campo *cond* da instrução, que impõe uma condição de execução sobre ela, e os dois bits seguintes, que compõem o campo *op0* e fazem parte do *opcode*, servem para

classificar o tipo da instrução. Essa classificação é importante porque, conforme descrito a seguir, os *opcodes* dos diferentes tipos de instruções variam em tamanho.

2.3.1 Instruções de Tipo D

As instruções de processamento de dados, ou de tipo D, realizam operações aritméticas, lógicas, de movimentação ou de comparação entre valores. Essas instruções estão listadas na Tabela 1.

Vale ressaltar que somente algumas instruções podem alterar as *flags* do CPSR, como as instruções ADDS, SUBS, RSB e CMP. Além disso, todas as instruções de tipo D têm suporte para imediatos no lugar do registrador RM.

Tabela 1 – Tipos, mnemônicos, op1 e descrições das instruções do tipo D.
Fonte: A autora (2025).

Tipo	**Mnemônico**	**op1**	**Descrição**
Aritméticas	ADD	00000	RD <= RN + RM/Im
	ADDS	00001	RD <= RN + RM/Im Altera *flags*
	SUB	00010	RD <= RN - RM/Im
	SUBS	00011	RD <= RN - RM/Im Altera *flags*
	RSB	00100	RD <= RM/Im - RN Altera *flags*
	MUL	00101	RD <= RN · RM/Im
	UDIV	00110	RD <= RN ÷ RM/Im
Lógicas	NOT	00111	RD <= RM/Im
	AND	01000	RD <= RN and RM/Im
	ORR	01001	RD <= RN or RM/Im
	EOR	01010	RD <= RN xor RM/Im
Movimentação	MOV	01011	RD <= RN/Im
Comparação	CMP	01100	RN - RM/Im Altera *flags*

A Tabela 2 mostra o formato das instruções de tipo D. O *opcode* tem tamanho de 8 bits e é dividido em *op0*, bit I e campo *op1*. Este último é próprio de cada instrução e é utilizado para identificar a operação realizada. Ele pode ser observado na Tabela 1, junto a cada instrução. Já o bit I é utilizado para identificar se a instrução deve interpretar a informação nos bits 9 a 0 como um registrador ou valor imediato.

Tabela 2 – Formato das instruções do tipo D. Fonte: A autora (2025).

Formato	*cond*	op0 (00)	I	op1	RN	RD	RM/Imediato
Bits	31:28	27:26	25	24:20	19:15	14:10	9:0

2.3.2 Instruções de Tipo B

2.3.3 Instruções de Tipo M

2.3.4 Instruções de Tipo A

2.4 Organização da Memória

3 Compilador: Fase de Análise

3.1 Modelagem

3.2 Análise Léxica

3.3 Análise Sintática

3.4 Análise Semântica

4 Compilador: Fase de Síntese

4.1 Modelagem

4.2 Geração de Código Intermediário

4.3 Geração de Código Assembly

4.4 Geração de Código Executável

4.5 Gerenciamento de Memória

5 Exemplos

5.1 Código Fonte

5.2 Código Intermediário

5.3 Código Assembly

5.4 Código Executável

5.5 ...

6 Conclusão

Referências